

C++ STL 容器、迭代器与算法详解

本文档旨在全面介绍 C++ 标准模板库（STL）中常用的容器、对应的迭代器类型、支持的操作与方法、适配器以及常用算法。本文将结合现代 C++ 的新特性，通过示例代码帮助理解和应用这些工具。

目录

1. STL 容器
 - 顺序容器
 - `std::array`
 - `std::vector`
 - `std::deque`
 - `std::list`
 - 关联容器
 - `std::set`
 - `std::map`
 - 无序容器
 - `std::unordered_set`
 - `std::unordered_map`
2. 迭代器
 - 迭代器类型
 - 常用迭代器操作
 - 迭代器注意事项
3. 适配器与迭代器适配器
 - 容器适配器
 - `std::stack`
 - `std::queue`
 - `std::priority_queue`
 - 迭代器适配器
4. 常用 STL 算法
 - `std::sort`
 - `std::find`
 - `std::for_each`
 - `std::transform`
 - `std::accumulate`
 - `std::copy`
 - `std::copy_if`
 - `std::copy_n`
 - `std::equal`
5. 示例代码
6. 总结

STL 容器

STL 容器是用于存储和管理数据的模板类，分为顺序容器、关联容器和无序容器三大类。每种容器适用于不同的场景，选择合适的容器能显著提升程序性能和代码可读性。

顺序容器

顺序容器按照元素的线性顺序存储数据，适合需要频繁插入、删除和顺序访问的场景。

std::array

- **描述：**固定大小的数组，封装了 C 风格数组的功能，提供更好的安全性和兼容 STL 算法。

- **迭代器类型：**随机访问迭代器。

- **主要操作：**

- size(), max_size()

- **参数：**无参数。

- **用法：**arr.size() 返回数组的大小。

- operator[], at()

- **参数：**索引 (size_type) 。

- **用法：**

```
arr[2] = 10;           // 使用下标访问
int value = arr.at(2); // 使用 at() 访问，带边界检查
```

- front(), back()

- **参数：**无参数。

- **用法：**

```
int first = arr.front(); // 获取第一个元素
int last  = arr.back();  // 获取最后一个元素
```

- fill(), swap()

- **参数：**

- fill(value)：填充所有元素为 value 。

- swap(other_array)：交换两个数组的内容。

- **用法：**

```
arr.fill(5);           // 将所有元素设置为 5
std::array<int, 5> other = {1, 2, 3, 4, 5};
arr.swap(other);        // 交换内容
```

- **注意事项：**

- 大小在编译时固定，无法动态调整。
- 支持大部分 STL 算法和范围基于循环。

std::vector

- **描述：**动态数组，支持快速随机访问和高效的尾部插入/删除操作。

- **迭代器类型：**随机访问迭代器。

- **主要操作：**

- push_back(), emplace_back()

- **参数：**

- push_back(value)：添加一个元素 value 到末尾。

- emplace_back(args...)：原地构造一个元素，传递构造参数。

- **用法：**

```
vec.push_back(10); // 添加元素 10
vec.emplace_back(20); // 添加元素 20，避免拷贝
```

- size(), capacity()

- **参数：**无参数。

- **用法：**vec.size() 返回当前元素数量，vec.capacity() 返回当前容量。

- operator[], at()

- **参数：**索引（size_type）。
- **用法：**

```
vec[1] = 15;           // 使用下标访问
int value = vec.at(1); // 使用 at() 访问，带边界检查
```

- **注意事项：**
 - 插入/删除操作可能导致内存重新分配，迭代器失效。
 - 使用 `emplace_back` 可以减少不必要的拷贝。

`std::deque`

- **描述：**双端队列，支持在两端高效插入和删除。
- **迭代器类型：**随机访问迭代器。
- **主要操作：**

- `push_back()`，`push_front()`
 - **参数：**
 - `push_back(value)`：在末尾添加元素 `value`。
 - `push_front(value)`：在开头添加元素 `value`。
 - **用法：**

```
deq.push_back(30); // 添加元素 30 到末尾
deq.push_front(10); // 添加元素 10 到开头
```

- `pop_back()`，`pop_front()`
 - **参数：**无参数。
 - **用法：**
- ```
deq.pop_back(); // 移除末尾元素
deq.pop_front(); // 移除开头元素
```

- `operator[]`，`at()`
    - **参数：**索引（size\_type）。
    - **用法：**
- ```
deq[2] = 25;           // 使用下标访问
int value = deq.at(2); // 使用 at() 访问，带边界检查
```

- **注意事项：**
 - 随机访问性能略低于 `std::vector`。
 - 适合需要频繁在两端操作的场景。

`std::list`

- **描述：**双向链表，支持在任意位置高效插入和删除。
- **迭代器类型：**双向迭代器。
- **主要操作：**

- `push_back()`，`push_front()`
 - **参数：**
 - `push_back(value)`：在末尾添加元素 `value`。
 - `push_front(value)`：在开头添加元素 `value`。
 - **用法：**

```
lst.push_back(50); // 添加元素 50 到末尾
lst.push_front(20); // 添加元素 20 到开头
```

- `insert()`，`erase()`
 - **参数：**
 - `insert(iterator pos, value)`：在指定位置 `pos` 前插入元素 `value`。

- `erase(iterator pos)`：移除指定位置 `pos` 的元素。
- **用法：**

```
auto it = lst.begin();
std::advance(it, 2);
lst.insert(it, 35); // 在第三个位置前插入 35
lst.erase(it);      // 移除第三个位置的元素
```

- `splice()`

- **参数：**
 - `splice(iterator pos, list& other, iterator it)`：将 `other` 列表中 `it` 指向的元素移动到当前列表的 `pos` 位置前。
- **用法：**

```
std::list<int> other = {60, 70};
auto it = other.begin();
lst.splice(lst.end(), other, it); // 将 other 的第一个元素移动到 lst 的末尾
```

- **注意事项：**
 - 不支持随机访问，遍历效率较低。
 - 适合需要频繁插入/删除但不需要随机访问的场景。

关联容器

关联容器基于键值对存储数据，通常以平衡二叉树实现，支持快速查找、插入和删除。

`std::set`

- **描述：**存储唯一键的有序集合。
- **迭代器类型：**双向迭代器。
- **主要操作：**
 - `insert()`, `erase()`
 - **参数：**
 - `insert(value)`：插入元素 `value`，如果已存在则不插入。
 - `erase(iterator pos)`：移除指定位置的元素。
 - `erase(key)`：移除所有键为 `key` 的元素（对于 `std::set`，是0或1）。
 - **用法：**

```
s.insert(100); // 插入元素 100
s.erase(100); // 移除元素 100
auto it = s.find(200);
if(it != s.end()) {
    s.erase(it); // 移除找到的元素
}
```

- `find()`, `count()`

- **参数：**
 - `find(key)`：查找键为 `key` 的元素，返回迭代器。
 - `count(key)`：返回键为 `key` 的元素数量（对于 `std::set`，是0或1）。
- **用法：**

```
auto it = s.find(300);
if(it != s.end()) {
    // 找到元素
}
int cnt = s.count(300); // cnt 为0或1
```

- `lower_bound()`, `upper_bound()`

- **参数：**
 - `lower_bound(key)`：返回不小于 `key` 的第一个元素的迭代器。

- `upper_bound(key)`：返回大于 `key` 的第一个元素的迭代器。
- **用法：**

```
auto it_low = s.lower_bound(250);
auto it_up = s.upper_bound(250);
```

- **注意事项：**
 - 元素自动排序，插入顺序不保证。
 - 不允许重复元素。
 - 不允许通过迭代器修改元素

`std::map`

- **描述：**存储键值对的有序集合，键唯一。
- **迭代器类型：**双向迭代器。
- **主要操作：**

- `insert()` , `erase()`

- **参数：**

- `insert({key, value})`：插入键值对，如果键已存在则不插入。
- `erase(iterator pos)`：移除指定位置的键值对。
- `erase(key)`：移除所有键为 `key` 的键值对（对于 `std::map`，是0或1）。

- **用法：**

```
m.insert({1, "one"});           // 插入键值对 (1, "one")
m.erase(1);                     // 移除键为1的键值对
auto it = m.find(2);
if(it != m.end()) {
    m.erase(it);                // 移除找到的键值对
}
```

- `find()` , `count()`

- **参数：**

- `find(key)`：查找键为 `key` 的元素，返回迭代器。
- `count(key)`：返回键为 `key` 的元素数量（对于 `std::map`，是0或1）。

- **用法：**

```
auto it = m.find(3);
if(it != m.end()) {
    // 找到键值对
}
int cnt = m.count(3); // cnt 为0或1
```

- `operator[]` , `at()`

- **参数：**

- `operator[](key)`：访问键为 `key` 的值，如果键不存在则插入默认值。
- `at(key)`：访问键为 `key` 的值，若键不存在则抛出异常。

- **用法：**

```
m[4] = "four";                  // 插入或更新键为4的值
try {
    std::string val = m.at(4); // 获取键为4的值
} catch(const std::out_of_range& e) {
    // 处理异常
}
```

- **注意事项：**
 - 使用 `operator[]` 可能会插入新元素。
 - 适合需要键值对应且有序的数据存储。
 - 不允许通过迭代器修改元素的 `key`

无序容器

无序容器基于哈希表实现，支持平均常数时间复杂度的查找、插入和删除操作。

std::unordered_set

- **描述：**存储唯一键的无序集合。
- **迭代器类型：**前向迭代器。
- **主要操作：**
 - insert(), erase()
 - **参数：**
 - insert(value)：插入元素 value，如果已存在则不插入。
 - erase(iterator pos)：移除指定位置的元素。
 - erase(key)：移除所有键为 key 的元素（对于 std::unordered_set，是0或1）。
 - **用法：**

```
us.insert(500);           // 插入元素 500
us.erase(500);           // 移除元素 500
auto it = us.find(600);
if(it != us.end()) {
    us.erase(it);        // 移除找到的元素
}
```
 - find(), count()
 - **参数：**
 - find(key)：查找键为 key 的元素，返回迭代器。
 - count(key)：返回键为 key 的元素数量（对于 std::unordered_set，是0或1）。
 - **用法：**

```
auto it = us.find(700);
if(it != us.end()) {
    // 找到元素
}
int cnt = us.count(700); // cnt 为0或1
```
 - bucket_count(), load_factor()
 - **参数：**无参数。
 - **用法：**

```
size_t buckets = us.bucket_count(); // 获取桶的数量
float load = us.load_factor();      // 获取负载因子
```
- **注意事项：**
 - 元素顺序不可预测。
 - 不允许重复元素。
 - 不允许通过迭代器修改元素

std::unordered_map

- **描述：**存储键值对的无序集合，键唯一。
- **迭代器类型：**前向迭代器。
- **主要操作：**
 - insert(), erase()
 - **参数：**
 - insert({key, value})：插入键值对，如果键已存在则不插入。
 - erase(iterator pos)：移除指定位置的键值对。
 - erase(key)：移除所有键为 key 的键值对（对于 std::unordered_map，是0或1）。
 - **用法：**

```
um.insert({10, "ten"});           // 插入键值对 (10, "ten")
um.erase(10);                     // 移除键为10的键值对
auto it = um.find(20);
if(it != um.end()) {
    um.erase(it);                 // 移除找到的键值对
}
```

◦ find(), count()

▪ **参数:**

- find(key): 查找键为 key 的元素, 返回迭代器。
- count(key): 返回键为 key 的元素数量 (对于 std::unordered_map, 是0或1)。

▪ **用法:**

```
auto it = um.find(30);
if(it != um.end()) {
    // 找到键值对
}
int cnt = um.count(30); // cnt 为0或1
```

◦ operator[], at()

▪ **参数:**

- operator[](key): 访问键为 key 的值, 如果键不存在则插入默认值。
- at(key): 访问键为 key 的值, 若键不存在则抛出异常。

▪ **用法:**

```
um[40] = "forty";                 // 插入或更新键为40的值
try {
    std::string val = um.at(40); // 获取键为40的值
} catch(const std::out_of_range& e) {
    // 处理异常
}
```

• **注意事项:**

- 元素顺序不可预测。
- 使用 operator[] 可能会插入新元素。
- 不允许通过迭代器修改元素的 key

迭代器

迭代器是用于遍历 STL 容器的对象, 提供了一种抽象化的方式来访问容器中的元素。

迭代器类型

根据功能不同, STL 提供了多种迭代器类型:

1. **输入迭代器 (Input Iterator)**: 只读访问, 一次前移一步。
2. **输出迭代器 (Output Iterator)**: 只写访问, 一次前移一步。
3. **前向迭代器 (Forward Iterator)**: 可读写, 支持多次遍历。
4. **双向迭代器 (Bidirectional Iterator)**: 可向前和向后遍历。
5. **随机访问迭代器 (Random Access Iterator)**: 支持任意方向和步长的移动。

常用迭代器操作

• **获取迭代器:**

◦ begin(), end()

- **参数:** 无参数。
- **用法:**

```
auto it = container.begin();
auto end_it = container.end();
```

◦ `cbegin()` , `cend()`

- **参数:** 无参数。

- **用法:**

```
auto cit = container.cbegin();
auto cend_it = container.cend();
```

◦ `rbegin()` , `rend()`

- **参数:** 无参数。

- **用法:**

```
auto rit = container.rbegin();
auto rend_it = container.rend();
```

◦ `crbegin()` , `crend()`

- **参数:** 无参数。

- **用法:**

```
auto crit = container.crbegin();
auto crend_it = container.crend();
```

- **遍历容器:**

```
for(auto it = container.begin(); it != container.end(); ++it) {
    // 访问 *it
}
```

- **范围基于循环** (现代 C++ 特性) :

```
for(auto& element : container) {
    // 访问 element
}
```

迭代器注意事项

- **迭代器失效:** 某些操作 (如 `vector` 的 `push_back`) 可能导致迭代器失效, 需谨慎处理。
- **类型匹配:** 确保迭代器类型与容器类型匹配, 避免类型错误。
- **避免悬挂迭代器:** 删除元素后, 相关迭代器可能成为悬挂迭代器, 应避免使用。

适配器与迭代器适配器

容器适配器

容器适配器是在现有容器之上构建的特定接口, 用于实现特定的数据结构。

`std::stack`

- **描述:** 后进先出 (LIFO) 数据结构。
- **底层容器:** 默认 `std::deque`, 也可使用 `std::vector` 或 `std::list`。
- **主要操作:**
 - `push()` , `pop()`
 - **参数:**
 - `push(value)`: 将元素 `value` 压入栈顶。
 - `pop()`: 移除栈顶元素。
 - **用法:**

```
stack.push(100); // 压入 100
stack.pop();     // 移除栈顶元素
```

- top()

- **参数：**无参数。
- **用法：**

```
int topElement = stack.top(); // 获取栈顶元素
```

- empty(), size()

- **参数：**无参数。
- **用法：**

```
bool isEmpty = stack.empty(); // 检查是否为空
size_t stackSize = stack.size(); // 获取栈大小
```

- **注意事项：**

- 仅支持在顶端进行插入和删除操作。

std::queue

- **描述：**先进先出（FIFO）数据结构。

- **底层容器：**默认 std::deque。

- **主要操作：**

- push(), pop()

- **参数：**
 - push(value)：将元素 value 插入队列尾部。
 - pop()：移除队列前端元素。
- **用法：**

```
queue.push(200); // 插入 200 到尾部
queue.pop();     // 移除前端元素
```

- front(), back()

- **参数：**无参数。
- **用法：**

```
int frontElement = queue.front(); // 获取前端元素
int backElement = queue.back();   // 获取尾部元素
```

- empty(), size()

- **参数：**无参数。
- **用法：**

```
bool isEmpty = queue.empty(); // 检查是否为空
size_t queueSize = queue.size(); // 获取队列大小
```

- **注意事项：**

- 仅支持在尾部插入和前端删除操作。

std::priority_queue

- **描述：**基于堆的优先级队列，元素按优先级顺序排列。

- **底层容器：**默认 std::vector。

- **主要操作：**

- push(), pop()

- **参数：**
 - push(value)：将元素 value 插入优先级队列。
 - pop()：移除最高优先级的元素。

- **用法:**

```
pq.push(300); // 插入 300
pq.pop();     // 移除最高优先级元素
```

- top()

- **参数:** 无参数。
- **用法:**

```
int topElement = pq.top(); // 获取最高优先级元素
```

- empty(), size()

- **参数:** 无参数。
- **用法:**

```
bool isEmpty = pq.empty(); // 检查是否为空
size_t pqSize = pq.size(); // 获取队列大小
```

- **注意事项:**

- 需要自定义比较器时，可通过模板参数指定。
- 默认情况下，使用 `std::less`，实现大顶堆。

迭代器适配器

迭代器适配器用于修改或限制迭代器的行为。

- **std::reverse_iterator** : 反向遍历容器。

- **用法:**

```
std::reverse_iterator<std::vector<int>::iterator> rit(vec.end());
for(; rit != std::reverse_iterator<std::vector<int>::iterator>(vec.begin()); ++rit) {
    std::cout << *rit << " ";
}
```

- **std::back_inserter** : 用于向容器尾部插入元素。

- **用法:**

```
std::copy(source.begin(), source.end(), std::back_inserter(destination));
```

- **std::front_inserter** : 用于向容器前端插入元素。

- **用法:**

```
std::copy(source.begin(), source.end(), std::front_inserter(destination));
```

- **std::inserter** : 用于在指定位置插入元素。

- **用法:**

```
std::copy(source.begin(), source.end(), std::inserter(destination, destination.begin()));
```

示例: 使用 `std::reverse_iterator` 反向遍历 `std::vector`

```

#include <vector>
#include <algorithm>
#include <iostream>

int main() {
    std::vector<int> vec = {1, 2, 3, 4, 5};
    std::reverse_iterator<std::vector<int>::iterator> rit(vec.end());

    std::cout << "Reverse order: ";
    for(; rit != std::reverse_iterator<std::vector<int>::iterator>(vec.begin()); ++rit) {
        std::cout << *rit << " ";
    }
    return 0;
}

```

常用 STL 算法

STL 提供了丰富的算法库，可以与各种容器和迭代器配合使用，提高代码的简洁性和效率。

std::sort

- **功能：**对元素进行排序。
- **适用容器：**支持随机访问迭代器的容器，如 `std::vector`、`std::deque`。
- **主要参数：**
 - `begin`，`end`：要排序的范围。
 - `comp`（可选）：自定义比较函数或 Lambda 表达式。
- **用法：**

```

std::sort(vec.begin(), vec.end()); // 默认升序排序
std::sort(vec.begin(), vec.end(), [](int a, int b) { return a > b; }); // 降序排序

```

- **示例：**

```

#include <vector>
#include <algorithm>
#include <iostream>

int main() {
    std::vector<int> vec = {5, 2, 8, 1, 3};
    std::sort(vec.begin(), vec.end()); // 升序排序

    std::cout << "Sorted vector: ";
    for(auto v : vec) std::cout << v << " ";
    return 0;
}

```

std::find

- **功能：**查找第一个匹配的元素。
- **适用容器：**所有支持输入迭代器的容器。
- **主要参数：**
 - `begin`，`end`：要查找的范围。
 - `value`：要查找的值。
- **用法：**

```

auto it = std::find(vec.begin(), vec.end(), 30);

```

- **示例：**

```

#include <vector>
#include <algorithm>
#include <iostream>

int main() {
    std::vector<int> vec = {10, 20, 30, 40};
    auto it = std::find(vec.begin(), vec.end(), 30);

    if(it != vec.end())
        std::cout << "Element found: " << *it << std::endl;
    else
        std::cout << "Element not found." << std::endl;

    return 0;
}

```

std::for_each

- **功能：**对每个元素执行指定操作。
- **适用容器：**所有支持输入迭代器的容器。
- **主要参数：**
 - begin, end : 要遍历的范围。
 - func : 要执行的函数或 Lambda 表达式。
- **用法：**

```

std::for_each(vec.begin(), vec.end(), print);
std::for_each(vec.begin(), vec.end(), [](int x) { std::cout << x << " "; });

```

- **示例：**

```

#include <vector>
#include <algorithm>
#include <iostream>

void print(int x) {
    std::cout << x << " ";
}

int main() {
    std::vector<int> vec = {1, 2, 3, 4, 5};
    std::for_each(vec.begin(), vec.end(), print);
    return 0;
}

```

现代 C++ 使用 Lambda 表达式：

```

std::for_each(vec.begin(), vec.end(), [](int x) { std::cout << x << " "; });

```

std::transform

- **功能：**对元素进行转换操作，并存储结果。
- **适用容器：**所有支持输入和输出迭代器的容器。
- **主要参数：**
 - begin, end : 要转换的范围。
 - out_begin : 输出的起始位置。
 - func : 转换函数或 Lambda 表达式。
- **用法：**

```
std::transform(vec.begin(), vec.end(), result.begin(), [](int x) { return x * x; });
```

- **示例:**

```
#include <vector>
#include <algorithm>
#include <iostream>

int main() {
    std::vector<int> vec = {1, 2, 3, 4, 5};
    std::vector<int> result(vec.size());

    std::transform(vec.begin(), vec.end(), result.begin(), [](int x) { return x * x; });

    std::cout << "Transformed vector: ";
    for(auto v : result) std::cout << v << " ";
    return 0;
}
```

std::accumulate

- **功能:** 对元素进行累加。
- **适用容器:** 所有支持输入迭代器的容器。
- **主要参数:**
 - begin, end : 要累加的范围。
 - init : 初始值。
 - func (可选) : 自定义累加函数。
- **用法:**

```
int sum = std::accumulate(vec.begin(), vec.end(), 0);
int product = std::accumulate(vec.begin(), vec.end(), 1, std::multiplies<int>());
```

- **示例:**

```
#include <vector>
#include <numeric>
#include <iostream>

int main() {
    std::vector<int> vec = {1, 2, 3, 4};
    int sum = std::accumulate(vec.begin(), vec.end(), 0);
    std::cout << "Sum: " << sum << std::endl;
    return 0;
}
```

std::copy

- **功能:** 复制元素到另一个容器或输出迭代器。
- **适用容器:** 所有支持输入和输出迭代器的容器。
- **主要参数:**
 - begin, end : 要复制的范围。
 - out_it : 目标位置的迭代器。
- **用法:**

```
std::copy(src.begin(), src.end(), dest.begin());
std::copy(src.begin(), src.end(), std::back_inserter(dest));
```

- **示例:**

```

#include <vector>
#include <algorithm>
#include <iostream>

int main() {
    std::vector<int> src = {1, 2, 3};
    std::vector<int> dest(3);

    std::copy(src.begin(), src.end(), dest.begin());

    std::cout << "Destination vector: ";
    for(auto v : dest) std::cout << v << " ";
    return 0;
}

```

std::copy_if

- **功能：**复制满足条件的元素到目标位置。
- **适用容器：**所有支持输入和输出迭代器的容器。
- **主要参数：**
 - begin, end : 要复制的范围。
 - out_it : 目标位置的迭代器。
 - pred : 谓词函数或 Lambda 表达式, 决定是否复制元素。
- **用法：**

```
std::copy_if(src.begin(), src.end(), std::back_inserter(dest), [](int x) { return x % 2 == 0; });
```

- **示例：**

```

#include <vector>
#include <algorithm>
#include <iostream>

int main() {
    std::vector<int> src = {1, 2, 3, 4, 5, 6};
    std::vector<int> dest;

    std::copy_if(src.begin(), src.end(), std::back_inserter(dest), [](int x) { return x % 2 == 0; });

    std::cout << "Even numbers: ";
    for(auto v : dest) std::cout << v << " ";
    return 0;
}

```

std::copy_n

- **功能：**从源序列复制指定数量的元素到目标位置。
- **适用容器：**所有支持输入和输出迭代器的容器。
- **主要参数：**
 - begin : 源序列的起始迭代器。
 - count : 要复制的元素数量。
 - out_it : 目标位置的迭代器。
- **用法：**

```
std::copy_n(src.begin(), 3, std::back_inserter(dest));
```

- **示例：**

```

#include <vector>
#include <algorithm>
#include <iostream>

int main() {
    std::vector<int> src = {10, 20, 30, 40, 50};
    std::vector<int> dest;

    std::copy_n(src.begin(), 3, std::back_inserter(dest));

    std::cout << "Copied elements: ";
    for(auto v : dest) std::cout << v << " ";
    return 0;
}

```

std::equal

- **功能**：检查两个序列是否相等。
- **适用容器**：所有支持输入迭代器的容器。
- **主要参数**：
 - first1, last1：第一个序列的范围。
 - first2：第二个序列的起始迭代器。
 - last2（可选）：第二个序列的尾后迭代器。
 - pred（可选）：自定义比较函数或 Lambda 表达式。
- **用法**：

```
bool isEqual = std::equal(vec1.begin(), vec1.end(), vec2.begin(), vec2.end());
```

- **示例**：

```

#include <vector>
#include <algorithm>
#include <iostream>

int main() {
    std::vector<int> vec1 = {1, 2, 3, 4};
    std::vector<int> vec2 = {1, 2, 3, 4};
    std::vector<int> vec3 = {1, 2, 3, 5};

    bool isEqual1 = std::equal(vec1.begin(), vec1.end(), vec2.begin(), vec2.end());
    bool isEqual2 = std::equal(vec1.begin(), vec1.end(), vec3.begin(), vec3.end());

    std::cout << "vec1 and vec2 are " << (isEqual1 ? "equal" : "not equal") << std::endl;
    std::cout << "vec1 and vec3 are " << (isEqual2 ? "equal" : "not equal") << std::endl;

    return 0;
}

```

示例代码

以下示例综合展示了容器、迭代器和算法的使用，体现了现代 C++ 的特性。

```

#include <iostream>
#include <vector>
#include <list>
#include <set>
#include <algorithm>
#include <numeric>
#include <iterator>
#include <array>

int main() {
    // 使用 std::array
    std::array<int, 5> arr = {5, 2, 8, 1, 3};
    std::sort(arr.begin(), arr.end(), [](int a, int b) { return a < b; });
    std::cout << "Sorted array: ";
    for(auto& v : arr) std::cout << v << " ";
    std::cout << std::endl;

    // 使用 std::vector 和 std::sort
    std::vector<int> vec = {5, 2, 8, 1, 3};
    std::sort(vec.begin(), vec.end(), [](int a, int b) { return a < b; });
    std::cout << "Sorted vector: ";
    for(auto& v : vec) std::cout << v << " ";
    std::cout << std::endl;

    // 使用 std::list 和 std::for_each
    std::list<int> lst = {10, 20, 30, 40};
    std::cout << "List elements: ";
    std::for_each(lst.begin(), lst.end(), [](int x) { std::cout << x << " "; });
    std::cout << std::endl;

    // 使用 std::set 和 std::find
    std::set<int> s = {3, 1, 4, 1, 5, 9};
    auto it = std::find(s.begin(), s.end(), 4);
    if(it != s.end())
        std::cout << "Found in set: " << *it << std::endl;
    else
        std::cout << "Element not found in set." << std::endl;

    // 使用 std::accumulate
    int sum = std::accumulate(vec.begin(), vec.end(), 0);
    std::cout << "Sum of vector elements: " << sum << std::endl;

    // 使用 std::transform
    std::vector<int> squared(vec.size());
    std::transform(vec.begin(), vec.end(), squared.begin(), [](int x) { return x * x; });
    std::cout << "Squared elements: ";
    for(auto& v : squared) std::cout << v << " ";
    std::cout << std::endl;

    // 使用 std::copy_if
    std::vector<int> even;
    std::copy_if(vec.begin(), vec.end(), std::back_inserter(even), [](int x) { return x % 2 == 0; });
    std::cout << "Even elements: ";
    for(auto& v : even) std::cout << v << " ";
    std::cout << std::endl;

    // 使用 std::copy_n
    std::vector<int> firstThree;
    std::copy_n(vec.begin(), 3, std::back_inserter(firstThree));
    std::cout << "First three elements: ";
    for(auto& v : firstThree) std::cout << v << " ";
}

```

```
std::cout << std::endl;

// 使用 std::equal
std::vector<int> vec2 = {1, 2, 3, 5, 8};
std::vector<int> vec3 = {1, 2, 3, 5, 8};
std::vector<int> vec4 = {1, 2, 3, 5, 9};

bool eq1 = std::equal(vec.begin(), vec.end(), vec2.begin());
bool eq2 = std::equal(vec.begin(), vec.end(), vec4.begin());

std::cout << "vec and vec2 are " << (eq1 ? "equal" : "not equal") << std::endl;
std::cout << "vec and vec4 are " << (eq2 ? "equal" : "not equal") << std::endl;

return 0;
}
```

输出：

```
Sorted array: 1 2 3 5 8
Sorted vector: 1 2 3 5 8
List elements: 10 20 30 40
Found in set: 4
Sum of vector elements: 19
Squared elements: 1 4 9 25 64
Even elements: 2 8
First three elements: 1 2 3
vec and vec2 are equal
vec and vec4 are not equal
```

总结

C++ 的 STL 提供了丰富的容器、迭代器和算法，极大地简化了数据处理和算法实现。通过合理选择容器和算法，并结合现代 C++ 的新特性（如 Lambda 表达式、范围基于循环等），可以编写出高效、简洁且易于维护的代码。理解 STL 的工作原理和使用场景，是掌握高效 C++ 编程的关键。

STL容器对左值和右值的支持

在C++的STL容器中，大多数方法（如 `push_back`、`emplace` 等）都经过精心设计，以支持左值和右值的重载，从而实现高效的值传递和存储。C++标准库通过模板的完美转发和重载机制，确保了容器操作的灵活性和性能。

常见容器方法对左值和右值的支持

以下是几种常见的STL容器操作及其对左值和右值的支持：

1. `push_back`（如 `std::vector` 和 `std::deque`）

`push_back` 在支持左值和右值方面表现出色：

- **左值**：拷贝构造对象并插入。
- **右值**：移动构造对象以避免不必要的拷贝。

示例代码：

```

#include <vector>
#include <iostream>
#include <string>

int main() {
    std::vector<std::string> vec;

    std::string str = "hello";
    vec.push_back(str); // 左值插入，调用拷贝构造函数
    vec.push_back("world"); // 右值插入，调用移动构造函数

    for (const auto& s : vec) {
        std::cout << s << "\n";
    }

    return 0;
}

```

2. `emplace_back` (如 `std::vector` 和 `std::deque`)

`emplace_back` 通过直接构造对象避免了不必要的拷贝或移动，是右值友好的方法：

- 接受参数并直接在容器内部构造对象。
- 支持完美转发（通过模板实现）。

示例代码：

```

#include <vector>
#include <iostream>

int main() {
    std::vector<std::pair<int, std::string>> vec;

    // 使用emplace_back直接构造对象
    vec.emplace_back(1, "hello");
    vec.emplace_back(2, "world");

    for (const auto& p : vec) {
        std::cout << p.first << ": " << p.second << "\n";
    }

    return 0;
}

```

3. `insert` (如 `std::vector` 和 `std::map`)

`insert` 同样支持左值和右值：

- **左值**：拷贝构造。
- **右值**：移动构造。

示例代码（以 `std::map` 为例）：

```

#include <map>
#include <iostream>

int main() {
    std::map<int, std::string> my_map;

    std::string value = "example";
    my_map.insert({1, value}); // 左值插入
    my_map.insert({2, "test"}); // 右值插入

    for (const auto& [key, val] : my_map) {
        std::cout << key << ": " << val << "\n";
    }

    return 0;
}

```

4. std::move 的使用

对于需要强制将左值转为右值的场景，可以使用 `std::move`：

```

#include <vector>
#include <iostream>
#include <string>

int main() {
    std::vector<std::string> vec;

    std::string str = "temporary";
    vec.push_back(std::move(str)); // 将左值转换为右值

    std::cout << "After move: " << str << "\n"; // str可能为空
    return 0;
}

```

STL如何实现对左值和右值的支持？

C++标准库利用以下技术对左值和右值进行优化支持：

1. 模板和完美转发

使用模板和 `std::forward`，将传递的参数类型（左值或右值）保留到最终调用的构造函数中。

2. 重载机制

针对左值和右值，方法可能有两种不同的重载形式。例如：

```

void push_back(const T& value); // 左值重载
void push_back(T&& value);     // 右值重载

```

3. 移动语义 (C++11引入)

通过移动构造函数和移动赋值操作符，实现高效的资源管理。

总结

STL容器的方法在设计时充分考虑了对左值和右值的支持，通过重载、模板和移动语义的结合，使得程序可以高效地处理不同类型的对象。如果有特殊需求（如避免拷贝），可以优先考虑使用 `emplace` 系列方法或者配合 `std::move` 来显式移动对象。